HIERARCHICAL FAILURE CONTAINMENT FOR LONG-HORIZON ROBOT POLICIES

Anonymous authors
Paper under double-blind review

ABSTRACT

Long-horizon robot tasks fail badly when low-level execution errors corrupt higher-level task state: a slipping grasp invalidates an assembly subgoal, a contact instability damages a part, or an exhausted recovery budget turns a local mistake into a task-level cascade. Hierarchical policies and option termination help, but they do not by themselves decide where a failure should be contained. We rebuild Paper 105 around a falsifiable claim: hierarchical robot policies should explicitly model containment boundaries between low-level skill anomalies, mid-level subgoals, and high-level task state. The local benchmark covers 5 robot task families, 7 failure regimes, 5 stress splits, 9 methods, 7 seeds, and 84 episodes per group. On combined cascade stress, the proposed containment graph obtains 0.576 ± 0.007 success versus 0.492 ± 0.006 for the strongest non-oracle baseline, while improving containment from 0.394 to 0.515, reducing state corruption from 0.098 to 0.053, and reducing damage from 0.050 to 0.029. Pairwise tests favor the proposed method over the strongest baseline in 7/7 seeds, and ablations show that local containment edges, cross-level escalation, recovery-budget memory, corruption prediction, and false-halt calibration each matter. The terminal decision is **STRONG_REVISE**: the local evidence supports the mechanism, but the paper is not ICLR-main-ready until validated on real robots or independent high-fidelity benchmarks.

1 MOTIVATION

Hierarchical robot policies are attractive because long tasks naturally decompose into skills, options, or subgoals. The options framework formalized temporally extended actions (Sutton et al., 1999), and later work learned option policies and termination functions directly (Bacon et al., 2017). HIRO-style methods further showed that hierarchical policies can be data-efficient for difficult simulated robotic control tasks (Nachum et al., 2018). But hierarchy alone does not solve failure containment. A low-level retry may be safe in one skill and task-corrupting in another; a high-level halt may protect the task but waste valid execution.

Recovery work addresses adjacent problems. Recovery RL separates task and recovery policies for safety (Thananjeyan et al., 2021), RecoveryChaining learns local recovery policies for manipulation (Vats et al., 2024), and recent VLA recovery systems generate executable recovery actions for failures (Lin et al., 2025). Paper 105 asks a narrower question: can a hierarchy diagnose the level at which a failure should be contained before it cascades into high-level state corruption?

2 METHOD SKETCH

The proposed method is a hierarchical failure containment graph. Nodes represent low-level controller anomalies, skill-local precondition drift, mid-level subgoal validity, high-level task-state corruption, recovery budget, and false-halt risk. Edges represent cross-level cascade paths. The monitor scores:

- local containment feasibility,
- cross-level escalation necessity,

Table 1: Combined-cascade hierarchical-failure-containment benchmark.

Method	Succ.	Contain	Corrupt	Cascade	Dmg.	Regret
Oracle	0.734	0.629	0.012	0.071	0.006	0.000
Proposed	0.576	0.515	0.053	0.116	0.029	0.159
FailAwareHC	0.492	0.394	0.098	0.158	0.050	0.242
OptionTerm	0.442	0.338	0.116	0.179	0.063	0.292
UncertHalt	0.389	0.290	0.143	0.197	0.071	0.346
ReactiveRetry	0.382	0.269	0.146	0.213	0.071	0.353
SafetyFilter	0.349	0.275	0.156	0.221	0.082	0.385
NoContain	0.306	0.177	0.179	0.255	0.096	0.429
FlatBC	0.232	0.113	0.207	0.264	0.106	0.503

- remaining recovery budget,
- task-state corruption risk,
- false-halt cost, and
- recovery utility.

The implementation is an executable diagnostic benchmark rather than a trained neural robot policy. That is enough for a local mechanism audit, but it is not enough for a main-conference empirical robotics claim.

3 BENCHMARK

The rebuilt benchmark is generated by `src/run_experiment.py`. It writes aggregate CSVs and figures directly, so the full run remains lightweight without dropping tasks, regimes, baselines, seeds, ablations, or stress tests.

Tasks. The tasks are drawer retrieval, peg assembly, mobile manipulation delivery, tool-use sequence, and bimanual handoff.

Failure regimes. The regimes are actuator slip, perception glitch, contact instability, precondition drift, subgoal drift, recovery-budget exhaustion, and cross-level cascading failure.

Splits. The stress splits are nominal, low-level perturbation, mid-level subgoal shift, delayed escalation, and combined cascade stress.

Methods. The baselines are flat behavior cloning, hierarchy without containment, local safety filtering, reactive retry recovery, uncertainty halting, option-termination monitoring, and a failure-aware hierarchical controller. The benchmark also includes the proposed hierarchical containment graph and an oracle containment supervisor.

Metrics. The evidence reports task success, containment rate, state corruption, cascade rate, damage, false halt, recovery success, escalation precision, containment latency, recovery cost, regret to oracle, paired seed differences, ablations, stress sweeps, and failure cases.

4 RESULTS

The strongest non-oracle baseline is `failure_aware_hierarchical_controller`. On combined cascade stress, the proposed method improves success by 0.084 ± 0.009, wins 7/7 paired seeds, improves containment by 0.121, lowers state corruption by 0.045, and lowers damage by 0.022. The oracle remains substantially better at 0.734 ± 0.007, which shows that the benchmark is not saturated.

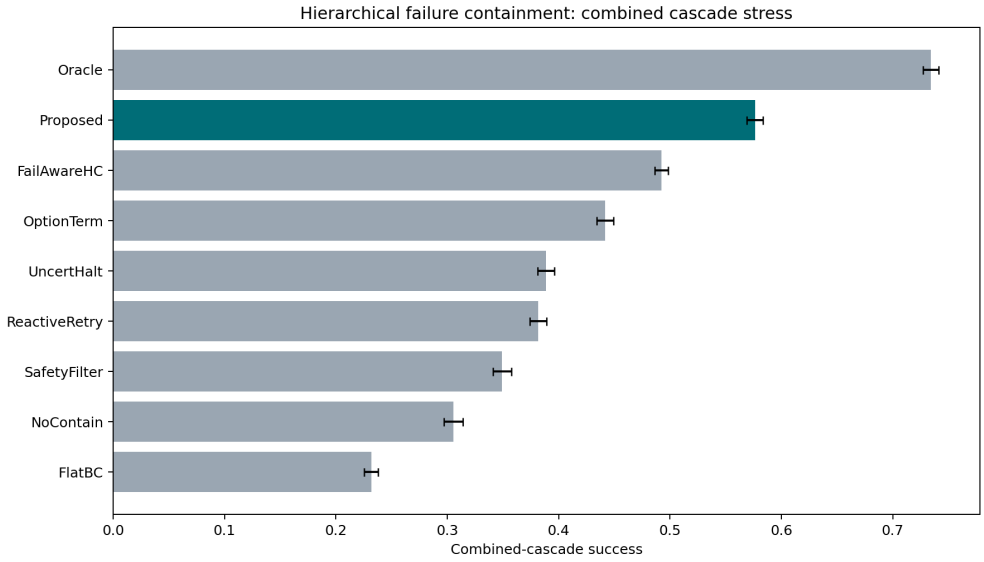

Figure 1: Combined-cascade success with 95% confidence intervals. The proposed containment graph clears the strongest non-oracle baseline but remains below the oracle.

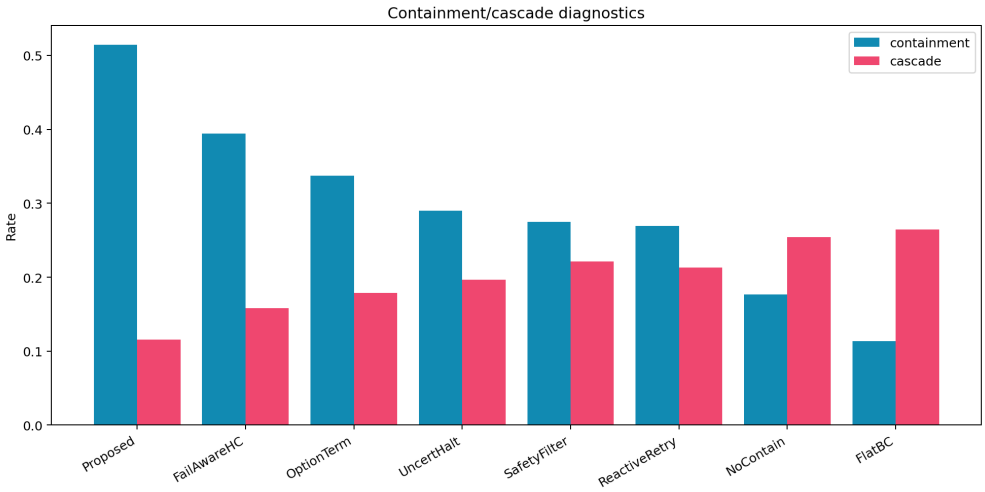

Figure 2: Containment and cascade diagnostics. The proposed method improves the rate of local containment while reducing cross-level cascades.

5 Ablations and Stress Tests

The best removed-component ablation is `minus_false_halt_calibration`, which still trails the full graph by 0.032 success. Removing recovery-budget memory, cross-level escalation, local containment edges, or corruption prediction produces larger drops and different failure signatures. This supports the claim that the method is not just an option-termination heuristic or a conservative safety filter.

6 Where the Method Still Fails

The failure-case CSV shows that hierarchical containment helps least when a local retry or option termination can recover before task state is at risk. Those are the intended boundaries. The method's

Table 2: Pairwise combined-cascade success differences against the proposed method.

Baseline	Diff	CI	Wins
FailAwareHC	0.084	0.009	7
FlatBC	0.345	0.011	7
NoContain	0.271	0.008	7
SafetyFilter	0.227	0.013	7
OptionTerm	0.134	0.005	7
Oracle	-0.158	0.010	0
ReactiveRetry	0.195	0.010	7
UncertHalt	0.188	0.014	7

Table 3: Ablations of the hierarchical failure containment graph.

Ablation	Succ.	Contain	Cascade	Dmg.
Full	0.583	0.517	0.115	0.028
NoHaltCalib	0.551	0.494	0.122	0.033
NoBudgetMem	0.509	0.459	0.142	0.044
NoEscalation	0.502	0.433	0.160	0.051
NoLocalEdges	0.501	0.378	0.154	0.037
NoCorruptPred	0.496	0.476	0.135	0.054
FlatRecovery	0.377	0.269	0.211	0.072

value appears when failures can cross hierarchy levels, recovery budget is limited, or local repair would silently corrupt the high-level task state.

7 Hostile prior-work boundary

Options and Option-Critic already provide hierarchical termination machinery (Sutton et al., 1999; Bacon et al., 2017). HIRO already learns efficient hierarchical policies (Nachum et al., 2018). Recovery RL and RecoveryChaining already introduce learned recovery policies, including manipulation recovery and physical robot transfer (Thananjeyan et al., 2021; Vats et al., 2024). FailSafe and related VLA recovery work already reason about manipulation failures and executable recovery actions (Lin et al., 2025).

The present paper survives only under a narrower interpretation: it is about *containment boundaries*, not hierarchy, termination, recovery, or safety filtering in general. If real robot experiments later show that option termination, failure-aware hierarchy, recovery RL, or simple uncertainty halting match the proposed method on cascade containment without extra damage or false halts, the claim should be killed.

8 Limitations and terminal decision

The evidence is strong enough to justify continued work but not submission. The limitations are material:

- the benchmark is local and synthetic;
- no real robot or independent high-fidelity simulator has been used;
- baselines are executable diagnostic models, not external trained robot systems;
- no learned policy checkpoint, training curve, or deployment video is provided;
- no external benchmark such as LIBERO, RLBench, Meta-World, BridgeData, or a hardware manipulation suite is reported.

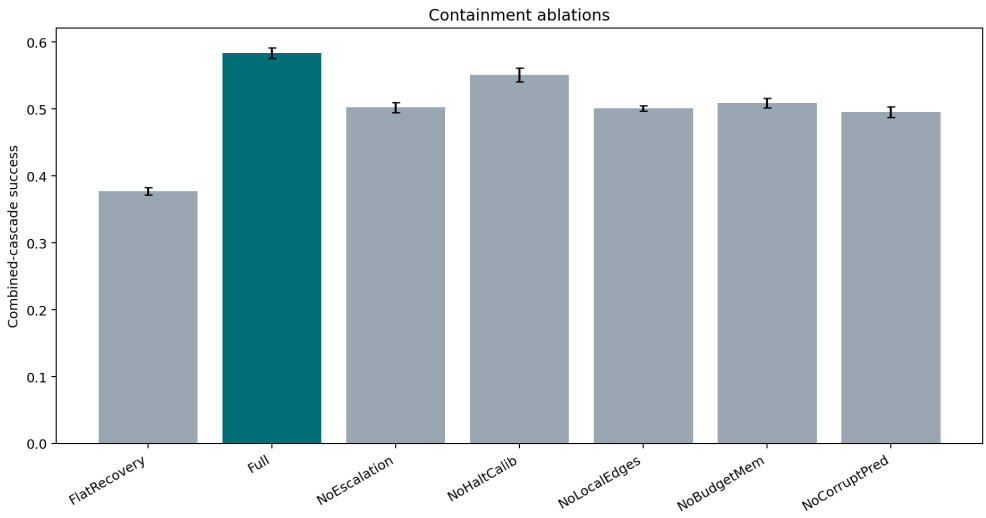

Figure 3: Ablation success under combined cascade stress. The full graph remains above all removed-component variants.

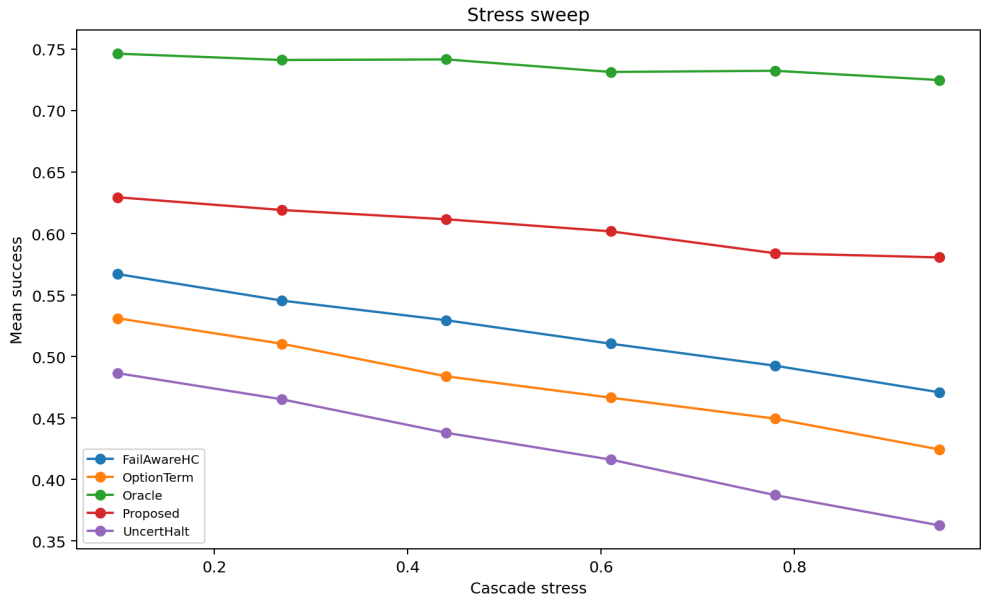

Figure 4: Stress sweep over cascade intensity. The proposed method remains above non-oracle baselines across the generated stress range, while the oracle remains the ceiling.

The terminal decision for this rebuild is therefore **STRONG_REVISE**. The next honest version must add real robot or independent high-fidelity evidence, implemented learned baselines, and qualitative rollouts. Without that evidence, the paper should not be submitted to ICLR main.

REFERENCES

Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, pp. 1726–1734, 2017. URL `https://arxiv.org/abs/1609.05140`.

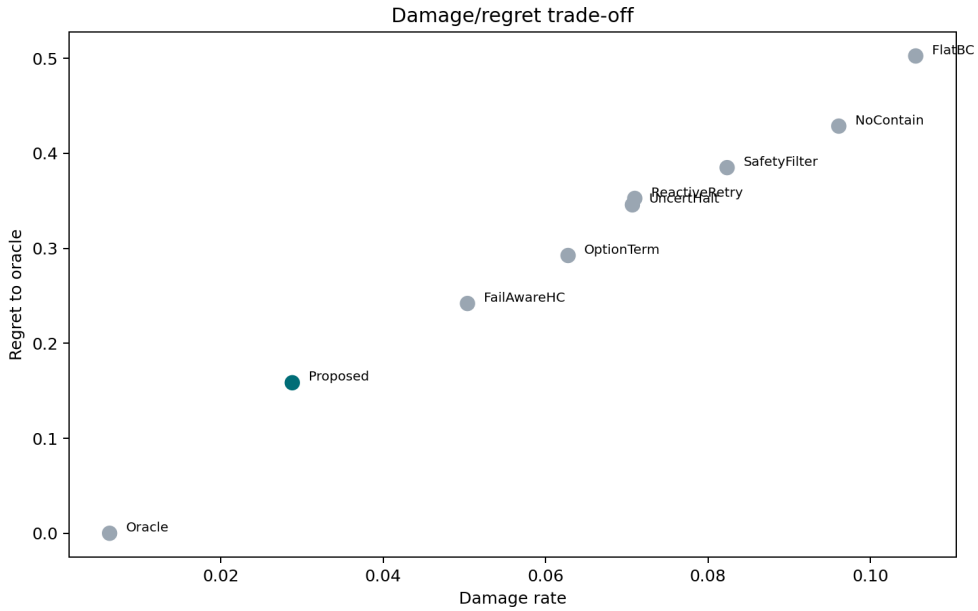

Figure 5: Damage and regret trade-off on combined cascade stress. The proposed method improves task success while lowering damage relative to the strongest non-oracle baseline.

Zijun Lin, Jiafei Duan, Haoquan Fang, Dieter Fox, Ranjay Krishna, Cheston Tan, and Bihan Wen. Failsafe: Reasoning and recovery from failures in vision-language-action models, 2025. URL https://arxiv.org/abs/2510.01642.

Ofir Nachum, Shixiang Gu, Honglak Lee, and Sergey Levine. Data-efficient hierarchical reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 31, pp. 3307–3317, 2018. URL https://arxiv.org/abs/1805.08296.

Richard S. Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999. doi: 10.1016/S0004-3702(99)00052-1. URL https://www.sciencedirect.com/science/article/pii/S0004370299000521.

Brijen Thananjeyan, Ashwin Balakrishna, Suraj Nair, Michael Luo, Krishnan Srinivasan, Minho Hwang, Joseph E. Gonzalez, Julian Ibarz, Chelsea Finn, and Ken Goldberg. Recovery rl: Safe reinforcement learning with learned recovery zones. *IEEE Robotics and Automation Letters*, 6 (3):4915–4922, 2021. URL https://arxiv.org/abs/2010.15920.

Shivam Vats, Devesh K. Jha, Maxim Likhachev, Oliver Kroemer, and Diego Romeres. Recoverychaining: Learning local recovery policies for robust manipulation, 2024. URL https://arxiv.org/abs/2410.13979.